Lab 10 — Praktijklab: de stack uitbaten

Doel: op de stack van lab 09 de logs en de metrieken lezen, limieten opleggen en een OOM uitlokken, een `HEALTHCHECK` in de image graveren (en ontdekken waarom Podman hem verliest), een container verharden, naar een privéregistry pushen, opruimen zonder te breken, en dan een Quadlet schrijven voor het starten bij het opstarten.

Vereisten — Labs 01 tot 09 afgewerkt: `~/labo-docker/09/` volledig, images `api:1.0`, `postgres:16-alpine`, `nginx:alpine`, `alpine`; podman-compose geïnstalleerd; op WSL `systemd=true` (nodig voor `podman stats` en voor stap 8).

Meegeleverde bestanden (`files/`) - `api.container` — de Quadlet van stap 8.

Stap 0 — De stack opnieuw starten

```
mkdir -p ~/labo-docker/10 && cd ~/labo-docker/09 && podman compose up -d ; sleep 12 ; cd
~/labo-docker/10
podman compose -f ~/labo-docker/09/compose.yaml ps 2>/dev/null | grep -c healthy
```

Observeer `2`: `db` en `api` zijn `healthy`, `web` heeft geen healthcheck.

Stap 1 — De logs lezen

```
podman logs -t --tail=2 stack_api_1
podman inspect stack_api_1 --format '{{.HostConfig.LogConfig.Type}}
{{.HostConfig.LogConfig.Path}}'
curl -s localhost:18080/api/ > /dev/null ; podman logs --tail=1 stack_web_1
```

Observeer twee regels met tijdstempel (`2026-…T12:58:00.068793040Z Arguments recus : (aucun)`); de driver `k8s-file` en een pad `…/overlay-containers/<id>/userdata/ctr.log` ; daarna de toegangsregel van nginx `"GET /api/ HTTP/1.1" 200 …` die je `curl` veroorzaakt heeft.

Uitleg. Elke container heeft zijn logbestand, geschreven door `conmon`. `-t` voegt de tijdstempel toe, `--since` / `--tail` begrenzen het lezen. Dat bestand heeft standaard geen maximumgrootte.

Stap 2 — Processen, resources, gebeurtenissen

```
podman top stack_api_1 user pid args
podman stats --no-stream --format 'table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}\t{{.MemPerc}}'
podman inspect stack_api_1 --format 'mem={{.HostConfig.Memory}} cpus={{.HostConfig.NanoCpus}}
pids={{.HostConfig.PidsLimit}}'
podman events --stream=false --since 5m --filter container=stack_api_1 | cut -c1-60
```

Observeer `podman 1 java -jar /app/api.jar` (gebruiker 1000 van de image, met jouw naam getoond); in `stats` enkele procent CPU en `27.58MB / 32.48GB`: de "limiet" is het geheugen van de machine; `mem=0 cpus=0 pids=0`: geen enkele limiet opgelegd; en de lijst van gebeurtenissen `container init`, `container start`, en dan om de vijf seconden een `health_status`.

Uitleg. `stats` leest de cgroups, `top` de processen, `inspect` de configuratie, `events` het journaal van de engine. Zonder `--memory` houdt niets deze container tegen vóór de hele machine.

Stap 3 — Begrenzen, en een OOM uitlokken

```
podman run -d --name api-limit --memory 256m --cpus 0.5 api:1.0 ; sleep 3
podman inspect api-limit --format 'mem={{.HostConfig.Memory}} cpus={{.HostConfig.NanoCpus}}'
podman stats --no-stream --format '{{.Name}} {{.MemUsage}} {{.MemPerc}}' api-limit
podman exec api-limit cat /sys/fs/cgroup/memory.max
podman rm -f -t 0 api-limit
```

Observeer `mem=268435456 cpus=500000000` (bytes en nano-CPU), `21.46MB / 268.4MB 8.00%`, en vanuit de container `memory.max` = `268435456`: de limiet is een cgroup-bestand dat het proces kan lezen.

```
podman run --name oom --memory 32m alpine sh -c 'head -c 100m /dev/zero | tail' ; echo
"exit=$?"
podman inspect oom --format 'oom={{.State.OOMKilled}} exit={{.State.ExitCode}}'
podman rm oom
```

Observeer `Killed`, `exit=137`, dan `oom=true exit=137`: `tail` wilde 100 MB in het geheugen houden binnen 32 MB, de kernel heeft het gedood.

```
podman run -d --name api-heap --memory 256m api:1.0 ; sleep 3
podman exec api-heap sh -c 'java -XX:+PrintFlagsFinal -version 2>/dev/null | grep "
MaxHeapSize"'
podman exec api-heap sh -c 'java -XX:MaxRAMPercentage=50 -XX:+PrintFlagsFinal -version
2>/dev/null | grep " MaxHeapSize"'
podman rm -f -t 0 api-heap
```

Observeer `MaxHeapSize = 132120576` (126 MB: onder een kleine limiet neemt de JVM standaard de helft, geen 25 %) en dan `134217728` (precies 128 MB met `MaxRAMPercentage=50`): de JVM heeft de limiet van de container gelezen en dimensioneert haar heap erop.

Uitleg. `--memory` schrijft `memory.max` in de cgroup van de container; overschrijden is geen applicatiefout maar een `SIGKILL` van de kernel. Voor Java stel je de heap in als percentage van die limiet, via `JAVA_TOOL_OPTIONS`.

Stap 4 — Een `HEALTHCHECK` in de image

```
printf 'FROM localhost/api:1.0\nHEALTHCHECK --interval=5s --timeout=3s --retries=3 CMD wget -
qO- http://localhost:8080/actuator/health || exit 1\n' > Dockerfile.health
podman build -q -f Dockerfile.health -t api:1.1 .
podman image inspect api:1.1 --format '{{.Config.Healthcheck}}'
podman build -q --format docker -f Dockerfile.health -t api:1.1 .
podman image inspect api:1.1 --format '{{.Config.Healthcheck.Test}}'
```

Observeer bij de eerste build een waarschuwing `HEALTHCHECK is not supported for OCI image format and will be ignored. Must use` docker `format`, dan `<nil>` — de instructie is verdwenen — en dan `[CMD-SHELL wget -qO- http://localhost:8080/actuator/health || exit 1]` na de build met `--format docker`.

```
podman run -d --name api-h api:1.1 ; sleep 9
podman ps --filter name=api-h --format '{{.Names}} {{.Status}}'
podman healthcheck run api-h ; echo "exit=$?"
podman inspect api-h --format '{{.State.Health.Status}} {{len .State.Health.Log}}'
podman exec api-h sh -c 'kill 1' ; sleep 1 ; podman ps -a --filter name=api-h --format
'{{.Status}}'
podman rm -f -t 0 api-h
```

Observeer `api-h Up 9 seconds (healthy)`, `exit=0`, `healthy` met al twee controles in de geschiedenis; en na `kill 1`, `Exited (143) 1 second ago (healthy)`: de healthcheck heeft niets herstart, het is een indicator — die zelfs zijn laatste toestand bewaart.

Uitleg. Het OCI-formaat heeft geen veld voor de healthcheck; Buildah laat hem vallen met een waarschuwing van één regel. Met `--format docker` draagt de image haar eigen probe, in rootless uitgevoerd door een systemd-timer.

Stap 5 — Verharden

```
podman run --rm --entrypoint id api:1.0
podman run --rm --entrypoint sh api:1.0 -c 'touch /app/x && echo "schrijven OK in /app"'
podman run --rm --read-only --tmpfs /tmp --entrypoint sh api:1.0 -c 'touch /app/x || echo
"alleen-lezen" ; touch /tmp/x && echo "tmp OK"'
```

Observeer `uid=1000(…) gid=1000(…)`: de image draait al als niet-root (`USER 1000:1000`, lab 05); `touch: cannot touch '/app/x': Permission denied` (de map is van root); en met `--read-only`, `Read-only file system` overal behalve in de `tmpfs`.

```
podman run --rm --entrypoint sh alpine -c 'chown nobody /etc/hostname && echo "chown OK (root
met capabilities)"'
podman run --rm --cap-drop ALL --entrypoint sh alpine -c 'chown nobody /etc/hostname'
podman run --rm --security-opt no-new-privileges --cap-drop ALL --read-only --tmpfs /tmp --user
1000 --entrypoint id alpine
```

Observeer `chown OK` voor de gewone root van de container, `chown: /etc/hostname: Operation not permitted` zonder capabilities, en `uid=1000 gid=0(root)` voor de volledige verhardingsregel — die zonder fout start.

Uitleg. Niet-root, alleen-lezen, nul capabilities, geen escalatie: vier opties, geen regel code. Die opties hebben hun exacte tegenhanger in Compose en in Kubernetes.

Stap 6 — Een privéregistry

```
podman run -d --name registry -p 5000:5000 docker.io/library/registry:2 ; sleep 2
podman tag api:1.0 localhost:5000/api:1.0
podman push localhost:5000/api:1.0 2>&1 | tail -1 | cut -c1-120
podman push --tls-verify=false localhost:5000/api:1.0 2>&1 | tail -1
curl -s localhost:5000/v2/_catalog ; echo ; curl -s localhost:5000/v2/api/tags/list ; echo
```

Observeer `Error: … Get "https://localhost:5000/v2/": http: server gave HTTP response to HTTPS client`, dan `Writing manifest to image destination`, en de API van de registry: `{"repositories":["api"]}` en `{"name":"api","tags":["1.0"]}`.

```
podman rmi localhost:5000/api:1.0
podman pull --tls-verify=false localhost:5000/api:1.0 2>&1 | tail -1
podman images --format '{{.Repository}}:{{.Tag}} {{.ID}}' | grep 'api:1.0'
podman image inspect api:1.0 --format '{{.Digest}}'
podman rm -f -t 0 -v registry
```

Observeer na de heen-en-terugreis `localhost:5000/api:1.0` en `localhost/api:1.0` met **hetzelfde** `IMAGE ID`; een digest `sha256:52b82ed9…` (de jouwe verschilt); en `rm -v`, want `registry:2` declareert een `VOLUME` (lab 06).

Uitleg. De registry slaat blobs op, geadresseerd op hun hash; wat eruit komt is bit voor bit wat erin ging. TLS wordt standaard geëist; `--tls-verify=false` is alleen aanvaardbaar voor deze test.

Stap 7 — Meten, dan opruimen

```
podman system df
podman images --filter dangling=true --format '{{.ID}} {{.Size}}'
podman image prune -f
podman system df | head -2
```

Observeer het overzicht per categorie met de kolom `RECLAIMABLE`; de lijst van `<none>`-images (bij jou wellicht de resten van de rebuilds van `api:1.1`); hun verwijderde ID's; en een totaal aan images dat daalt. De volumes en de containers zijn niet aangeraakt.

Uitleg. `image prune` zonder `-a` verwijdert alleen de dangling images: het is de enige `prune` die je uit reflex start. De andere (`-a`, `container`, `volume`, `system`) beslis je na lezing van `system df` en `volume ls --filter dangling=true`, en een volume verwijder je **bij naam**.

Stap 8 — Starten bij het opstarten met Quadlet (WSL)

Deze stap vereist systemd in je distributie (`systemd=true` in `/etc/wsl.conf`); ze kan niet herspeeld worden in de sandbox waarin dit lab geschreven is, de uitvoer hieronder wordt dus beschreven, niet aangehaald.

```
mkdir -p ~/.config/containers/systemd
cp <pad-naar-het-lab>/files/api.container ~/.config/containers/systemd/
systemctl --user daemon-reload
systemctl --user start api
systemctl --user status api --no-pager | head -5
podman ps --format '{{.Names}} {{.Status}} {{.Ports}}'
curl -s localhost:18080/actuator/health ; echo
journalctl --user -u api -n 3 --no-pager
```

Observeer een dienst `api.service` `active (running)`, een container `systemd-api` (voorvoegsel toegevoegd door Quadlet) gepubliceerd op `18080`, `{"status":"UP"}`, en de logs van de API in het systemd-journaal.

```
systemctl --user enable api
loginctl enable-linger $USER
systemctl --user stop api
podman ps -a --format '{{.Names}}' | grep -c systemd-api
```

Observeer `0` na `stop`: Quadlet start de container met `--rm`, het stoppen van de dienst verwijdert hem; bij de volgende start van de sessie (of van WSL, zodra de distributie gestart is) maakt systemd hem opnieuw aan.

Uitleg. Het `.container`-bestand wordt vertaald naar een systemd-unit; het is systemd, herstart door het systeem, dat Podman herstart. `enable-linger` houdt de `--user`-sessie in stand zonder open aanmelding. Laat voor de volgende labs de dienst **gestopt** (`systemctl --user disable --now api`) zodat poort 18080 vrij blijft.

Opruimen

```
cd ~/labo-docker/09 && podman compose down && cd ~/labo-docker/10
podman rmi api:1.1 localhost:5000/api:1.0 docker.io/library/registry:2
rm -f Dockerfile.health
podman ps -a --format '{{.Names}}' ; podman volume ls --format '{{.Name}}' ; podman images --
format '{{.Repository}}:{{.Tag}}'
```

Observeer geen enkele container, het volume `stack_pgdata` bewaard (de database van lab 09), en de images van de stack nog aanwezig. Einde van het Docker-deel: het Kubernetes-deel (lab 11) hergebruikt `api:1.0`, `nginx:alpine` en `postgres:16-alpine`.

Wat je nu moet kunnen beweren

- `logs -t`, `top`, `stats`, `events`: het bestand `ctr.log`, het proces `java` als UID 1000, `27 MB` gebruikt zonder limiet, om de vijf seconden een `health_status`.
- `--memory 256m` = `memory.max` leesbaar vanuit de container; `32m` voor 100 MB = `Killed`, `137`, `OOMKilled: true`; de JVM dimensioneert haar heap op de limiet, `MaxRAMPercentage` bepaalt het aandeel.
- `HEALTHCHECK` verloren in OCI-formaat (`<nil>`), bewaard met `--format docker` (`(healthy)`), en hij herstart niets (`Exited (143)`).

- `--read-only` + `--tmpfs`, `--cap-drop ALL` (`Operation not permitted`), `no-new-privileges`, niet-root: de verharde regel start.
- `push` geweigerd zonder TLS, aanvaard met `--tls-verify=false`; hetzelfde `IMAGE ID` na de heen-en-terugreis; `rm -v` voor de registry.
- Eerst `system df`, `image prune` (dangling) uit reflex, de rest bij naam.
- Quadlet: `.container` → `api.service`, `enable` + `enable-linger` om `wsl --shutdown` te overleven.